Section - A

(1)

(a) Explain the basic concept of class and object give proper example mark 2

Solve: A class is a blueprint or a template for creating objects, and an object is an instance of a class. Classes are used to define the properties and behaviors that objects of the class should have.

```
public class Puppy {
  int puppyAge;

  public Puppy(String name) {
    System.out.println("Name chosen is :" + name );
  }

  public void setAge( int age ) {
    puppyAge = age;
  }

  public int getAge( ) {
    System.out.println("Puppy's age is :" + puppyAge );
    return puppyAge;
  }

  public static void main(String []args) {
    Puppy myPuppy = new Puppy( "tommy" );

    myPuppy.setAge( 2 );

    myPuppy.getAge( );

    System.out.println("Variable Value :" + myPuppy.puppyAge );
  }
}
```

(b) What will be the Output of the below code: 2.5

```
public class A {
   public static void main(String[ ] args)
   {
      System.out.println('j' + 'a' + 'v' + 'a');
   }
}
```

Solve: 418

(b) **What is the concept of access specifiers when should we use this? Give proper example 3**

Solve: Access specifiers control the visibility of classes, methods, and variables. There are four types:

Public (public): Accessible from any class.
Private (private): Accessible only within the same class.
Protected (protected): Accessible within the same package and by subclasses.
Default (Package-Private): Accessible only within the same package.

Example:

```
public class Person {
   public String name;
   private int age;
   protected void celebrateBirthday() { /* ... */ }
   void sayHello() { /* ... */ }
}

public class Main {
   public static void main(String[] args) {
      Person person = new Person();
      person.name = "John"; // Public variable
      // person.age = 25; // Compilation error - private variable
      person.celebrateBirthday(); // Protected method
      person.sayHello(); // Default method
   }
}
```

(c) What are the limitations of object oriented programming mark 2.5

Limitaions of OOP :

1. The length of the programs developed using OOP language is much larger than the procedural approach. Since the program becomes larger in size, it requires more time to be executed that leads to slower execution of the program

2. We can't apply OOP everywhere as it is not a universal language. It is applied only when it is required. It is not suitable for all types of problems.

3. Programmers need to have brilliant designing skill and programming skill along with proper planning because using OOP is little bit tricky.

4. OOPs take time to get used to it. The thought process involved in object-oriented programming may not be natural for some people.

(2)

(a) Write down the differences between the Constructor and the method in Java mark 1.5

Solve:

1. Invocation:
Constructor: Automatically invoked during object creation.
Method: Explicitly invoked using the method name.

2. Return Type:
Constructor: No return type.
Method: Has a return type, which can be any valid data type or void.

3. Name:
Constructor: Has the same name as the class.
Method: Can have any name that is a valid identifier.

(b) What are the various types of Constructors mark 2

Solve:

Non Parameterized Constructor:
Automatically provided by Java if no constructor is defined.
Has no parameters and initializes instance variables to default values.

Parameterized Constructor:
Accepts parameters to initialize instance variables with provided values.
Allows for object customization during creation.

Copy Constructor:
Creates a new object by copying the values of another object.
Used for creating a new object with the same state as an existing one.

(c) Define constructor and destructor can you overload the Constructor or the destructor in a class? Give example mark 2.5

Solve :

Constructor:
A constructor is a special method in a class that initializes an object. In Java, constructors are automatically called when an object is created.

Destructor:
In Java, there is no explicit destructor. Garbage collection manages memory, releasing resources when objects are no longer referenced.

Constructor Overloading:
Yes, constructors can be overloaded in Java. It allows defining multiple constructors with different parameter lists for object initialization.

Example:

```
public class MyClass {
   public MyClass() { /* Default constructor */ }
   public MyClass(int value) { /* Parameterized constructor 1 */ }
   public MyClass(String stringValue) { /* Parameterized constructor 2 */ }
}
```

(d) What are the four main features of object oriented programming explain with proper example 4

Solve: The four main features of Object-Oriented Programming (OOP) are:

Encapsulation:

Explanation: Bundles data and methods into a class, hiding internal details.
Example:

```
public class Car {
   private String model;
   private int year;
   // Methods...
}
```

Inheritance:

Explanation: Allows a class to inherit properties and behaviors from another class.
Example:

```
public class Dog extends Animal {
   // Methods specific to Dog...
}
```

Polymorphism:

Explanation: Enables treating objects as instances of their parent class, supporting method overloading and overriding.
Example:

```
public int add(int a, int b) { /* ... */ }
public double add(double a, double b) { /* ... */ }

// Method Overriding
public void draw() { /* ... */ }
```

Abstraction:

Explanation: Simplifies systems by modeling classes based on essential properties and behaviors.
Example:

```
public abstract class Shape {
   public abstract void draw();
   public void move() { /* ... */ }
}
```

(3)

(a) Describe different types of inheritance with example 3

Single Inheritance:

Explanation: A class can inherit from only one superclass.
Example:

```
public class Animal { /* ... */ }

public class Dog extends Animal { /* ... */ }
```

Multiple Inheritance (Interface-based):

Explanation: A class can implement multiple interfaces, achieving a form of multiple inheritance in Java.
Example:

```
public interface Swimming { /* ... */ }

public interface Flying { /* ... */ }

public class Bird implements Swimming, Flying { /* ... */ }
```

Multilevel Inheritance:

Explanation: A class is derived from another class, and that derived class is used as the base class for another class.
Example:

```
public class Animal { /* ... */ }

public class Mammal extends Animal { /* ... */ }

public class Dog extends Mammal { /* ... */ }
```

(b) What is a diamond Problem? Consider the following code snippet where class Child tries to inherit both classes Parent1 and Parent2. What are the similarities between a diamond problem and the following code snipped. 3

```
//  Class 1
// First Parent class
class Parent1 {
  // ….
}
// Class 2
// Second Parent Class
class Parent2 {
  // …
}
// Class 3
// Trying to be child of both the classes
class Child extends Parent1, Parent2 {
  // Main driver method
  public static void main(String args[]) {
    //…..
  }
}
```

Solve: The diamond problem happens in programming when a class inherits from two classes that share a common parent, causing confusion.

The Java code tries this with the Child class inheriting from both Parent1 and Parent2, even though Java doesn't allow multiple inheritance directly. This is like a reminder to be careful, as allowing such things could lead to confusion in how the program works. Java handles this by using interfaces for multiple inheritance instead of classes, making things clearer.

So, the code snippet above would result in a compilation error.

(c) Rewrite the above code to successfully implement Multiple Inheritance in Java. 4

Solve:

In Java, direct multiple inheritance for classes is not supported due to the diamond problem. However, you can achieve a form of multiple inheritance using interfaces. The provided example demonstrates a Child class implementing both Parent1 and Parent2 interfaces, allowing it to inherit from both "parents" through interfaces rather than classes.

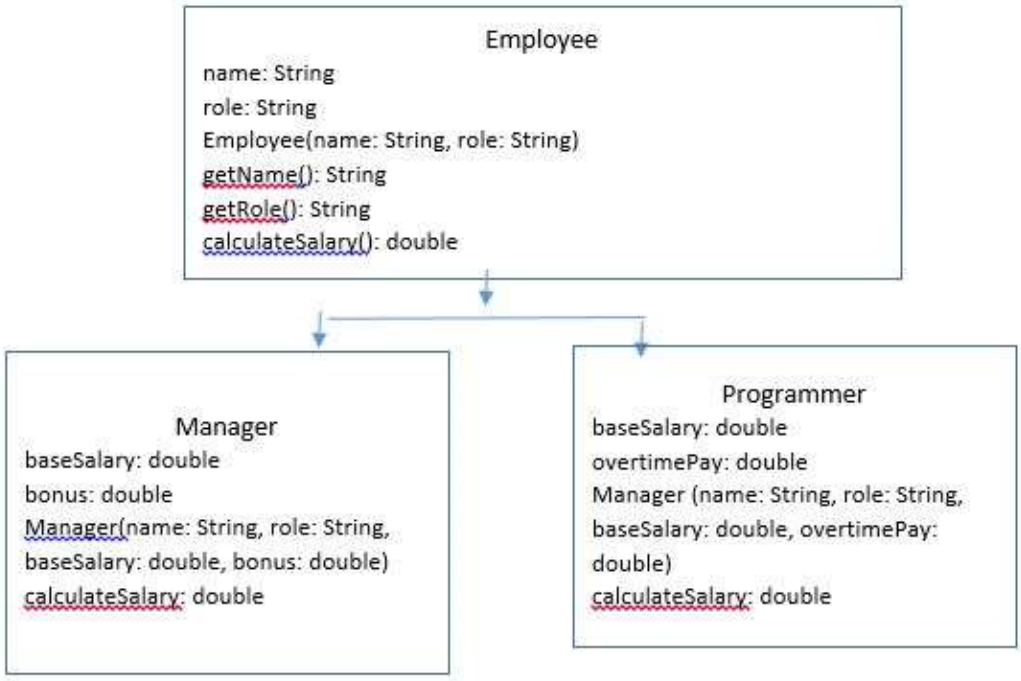
```
// Interface 1
// First Parent interface
interface Parent1 {
// ….}

// Interface 2
// Second Parent interface
interface Parent2 {
  // …. }

// Interface 3
// This time Interfaces will implement successfully
class Child implements Parent1, Parent2 {
          // …. }
public class Main {
 // Main driver method
        public static void main(String[] args) {
            // ….
         }
      }
```

(4)

(a) Write a Java program to create a class Employee with a method called calculateSalary(). Create two subclasses Manager and Programmer. In each subclass, override the calculateSalary () method to calculate and return the salary based on their specific roles. 5



Solve: class Employee {

```
    private String name;
    private String role;

    public Employee(String name, String role) {
        this.name = name;
        this.role = role;
    }

    public double calculateSalary() {
        return 0.0;
    }
}

class Manager extends Employee {
    private double baseSalary;
    private double bonus;

    public Manager(String name, double baseSalary, double bonus) {
```

```
        super(name, "Manager");
        this.baseSalary = baseSalary;
        this.bonus = bonus;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + bonus;
    }
}

class Programmer extends Employee {
    private double baseSalary;
    private double overtimePay;

    public Programmer(String name, double baseSalary, double overtimePay) {
        super(name, "Programmer");
        this.baseSalary = baseSalary;
        this.overtimePay = overtimePay;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + overtimePay;
    }
}

public class Main {
    public static void main(String[] args) {
        Manager manager = new Manager("John Doe", 50000.0, 10000.0);
        Programmer programmer = new Programmer("Jane Smith", 60000.0, 5000.0);

        System.out.println("Manager's Salary: $" + manager.calculateSalary());
        System.out.println("Programmer's Salary: $" + programmer.calculateSalary());
    }
}
```

(b) Describe different types of Polymorphism 2.5
Solve:

Compile-time Polymorphism (Static):
Method Overloading: Using the same name for methods, but they behave differently based on the provided information.

Runtime Polymorphism (Dynamic):
Method Overriding: When a subclass provides its own version of a method from its superclass.
Polymorphic Reference: A reference that can point to different objects and behaves based on the actual object type.

Interfaces: A common set of rules that different classes can follow, providing a shared way to interact with them.

(c) Write down the difference of an Abstract class and an Interface 2.5
Solve:

Abstract Class	Interface
It can have constructors.	It cannot have constructors.
Can have both abstract and concrete methods.	Can only have abstract methods (until Java 8), and default/static
Can have fields (variables).	Can only have constant fields.
Supports access modifiers for methods.	All methods are implicitly public.

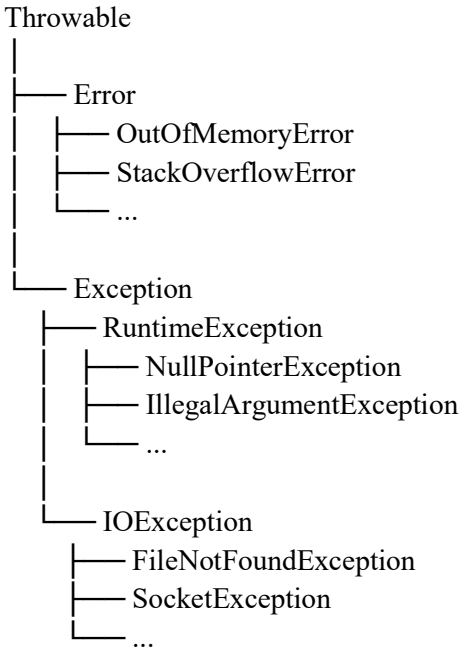
SECTION-B

(5)

(a) What are the differences between Error and Exception mark 2
Solve:

Error	Exception
Indicates serious issues in the JVM	Unusual situations in the program.
Usually not handled by regular programs.	Meant to be handled by the program.
Often not recoverable, may require restarting.	Can be recovered from by handling appropriately.
Unchecked, usually unexpected and severe.	Can be checked (expected) or unchecked (unexpected).

(c) Write down Exception Hierarchy 1.5
Solve:



In this hierarchy:

Throwable is the root class for all exceptions.
Error represents serious issues in the Java Virtual Machine (JVM).
Exception is a more general category for exceptional conditions in the program.
RuntimeException is a subclass of Exception and includes common runtime issues.
IOException is a subclass of Exception and includes common input/output issues.

(c) A palindromic number, also referred to as a numeral palindrome or numeric palindrome, is a number, like 1994991, that retains its value when its digits are reversed.
Develop a Java program that reads a series of numbers from "input.txt" and throws an exception if any of the numbers are not Palindrome mark 3.5

Solve:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

class NonPalindromeException extends Exception {
   public NonPalindromeException(String message) {
      super(message);
   }
}

public class PalindromeChecker {
   public static void main(String[] args) {
```

```
        try {
            checkPalindromesFromFile("input.txt");
        } catch (IOException | NonPalindromeException e) {
            System.err.println(e.getMessage());
        }
    }

    public static void checkPalindromesFromFile(String filename) throws IOException,
NonPalindromeException {
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {

reader.lines().mapToInt(Integer::parseInt).forEach(PalindromeChecker::validatePalindrome);
        }
    }

    public static void validatePalindrome(int number) throws NonPalindromeException {
        if (number != reverseNumber(number)) {
            throw new NonPalindromeException("Non-palindrome number found: " + number);
        }
    }

    public static int reverseNumber(int number) {
        int reversed = 0;
        while (number > 0) {
            int digit = number % 10;
            reversed = reversed * 10 + digit;
            number /= 10;
        }
        return reversed;
    }
}
```

(d) Explain finally and final keywords with proper examples. 2

Solve: finally Keyword:

The finally block in Java ensures that specific code is executed, whether an exception occurs or not.

final Keyword:

The final keyword is used to declare constants, make variables unmodifiable, or prevent method overriding. It's commonly employed for creating constants (final int MAX_VALUE = 100;), making variables unmodifiable within a class (final int myNumber = 42;)

(e) Give two examples of checked and unchecked exceptions. 1

Solve:

Checked Exceptions:

IOException.

SQLException.

Unchecked Exceptions:

NullPointerException.

ArrayIndexOutOfBoundsException.

(a) Explain usage of class FileInputStream by giving an example. (2.5)
Solve:
The FileInputStream class in Java is used to read data from a file. It is commonly employed for reading byte-oriented data, such as images or text files.

Example:

```
import java.io.FileInputStream;
import java.io.IOException;

public class FileInputStreamExample {
   public static void main(String[] args) {
      try (FileInputStream fis = new FileInputStream("example.txt")) {
         int byteRead;
         while ((byteRead = fis.read()) != -1) {
            System.out.print((char) byteRead); // Assuming it's a text file
         }
      } catch (IOException e) {
         System.err.println("IOException: " + e.getMessage());
      }
   }
}
```

In this example, the FileInputStream is used to read bytes from the "example.txt" file. It reads one byte at a time until the end of the file is reached (read() returns -1). The read bytes are then printed, assuming it's a text file. The try-with-resources statement is used to automatically close the FileInputStream when done.

(b) Describe bite streams character streams and standard streams with example Mark 3
Solve:

Byte Streams:
For raw binary data.

Example - FileInputStream:

```
import java.io.FileInputStream; import java.io.IOException;

public class ByteStreamExample {
   public static void main(String[] args) {
      try (FileInputStream fis = new FileInputStream("example.txt")) {
         int b; while ((b = fis.read()) != -1) { System.out.print((char) b); }
```

```
        } catch (IOException e) { System.err.println("IOException: " + e.getMessage()); }
    }
}
```

Character Streams:
For character data, suitable for text.

Example - FileReader:

```
import java.io.FileReader; import java.io.IOException;

public class CharacterStreamExample {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("example.txt")) {
            int c; while ((c = reader.read()) != -1) { System.out.print((char) c); }
        } catch (IOException e) { System.err.println("IOException: " + e.getMessage()); }
    }
}
```

Standard Streams:
For console interaction.

Example - System.in:

```
import java.util.Scanner;

public class StandardStreamExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter something: ");
        String userInput = scanner.nextLine();
        System.out.println("You entered: " + userInput);
    }
}
```

In these concise examples, byte streams handle binary, character streams handle text, and standard streams interact with the console.

(c) Write a Java program to compare two files lexicographically. Mark 4.5

According to Wikipedia:
In mathematics, the lexicographic or lexicographical order (also known as lexical order, dictionary order, alphabetical order or lexicographic(al) product) is a generalization of the way the alphabetical order of words is based on the alphabetical order of their component letters. This generalization consists primarily in defining a total order over the sequences (often called words in computer science) of elements of a finite totally ordered set, often called alphabet.

Solve: Below is a simple Java program that compares two files lexicographically:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class FileComparator {
   public static void main(String[] args) {
      String file1 = "file1.txt";
      String file2 = "file2.txt";

      try {
         if (compareFilesLexicographically(file1, file2) == 0) {
            System.out.println("Files are lexicographically equal.");
         } else {
            System.out.println("Files are lexicographically not equal.");
         }
      } catch (IOException e) {
         System.err.println("IOException: " + e.getMessage());
      }
   }

   public static int compareFilesLexicographically(String filePath1, String filePath2) throws IOException
{
      try (BufferedReader reader1 = new BufferedReader(new FileReader(filePath1));
           BufferedReader reader2 = new BufferedReader(new FileReader(filePath2))) {

         String line1, line2;
         while ((line1 = reader1.readLine()) != null && (line2 = reader2.readLine()) != null) {
            int comparison = line1.compareTo(line2);
            if (comparison != 0) {
               return comparison;
            }
         }

         // If one file is longer than the other
         if ((line1 != null && reader1.readLine() != null) || (line2 != null && reader2.readLine() != null)) {
            return -1; // Files have different lengths
         }

         return 0; // Files are lexicographically equal
      }
   }
}
```

This program compares two text files (file1.txt and file2.txt) lexicographically by reading them line by line and comparing each line. If the files are lexicographically equal, it prints a message indicating that. If they are not equal, it prints a different message. The compareFilesLexicographically method returns 0 if the files are equal, a negative value if the first file is lexicographically smaller, and a positive value if the first file is lexicographically larger.

(7)

(a) Write down the differences between Java Swing, Java AWT and JavaFX. Which one is better and why? 4

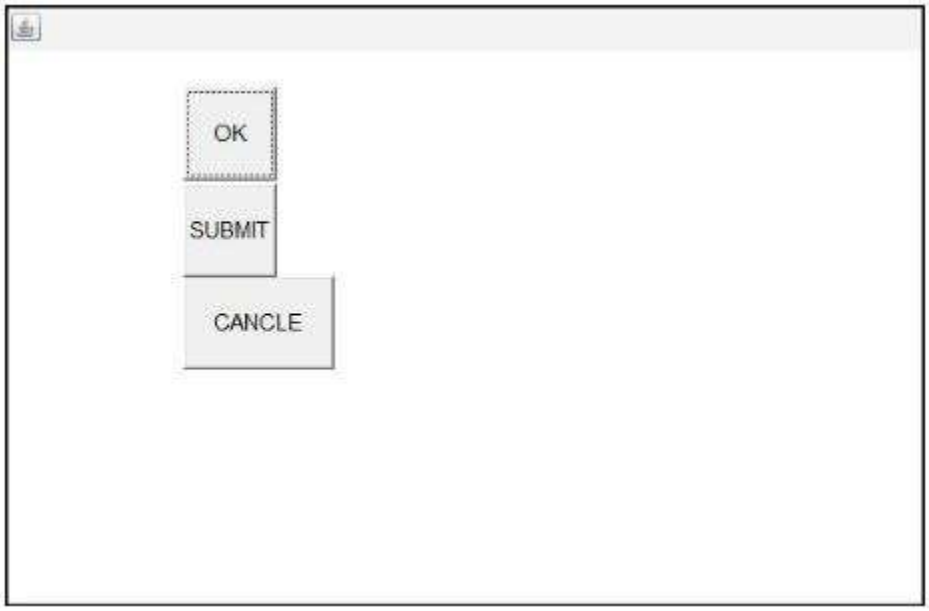
Feature	Swing	AWT	JavaFX
Architecture	MVC	Native Toolkit	Scene Graph
Components	Rich, Advanced	Basic, Platform-Dependent	Modern, Comprehensive
Look and Feel	Pluggable	Native	Modern, Customizable
Graphics	2D	Native Libraries	Scene Graph, Retained Mode
Event Handling	Separate Thread	Delegation Model	Event-Driven, Properties
Threading	Not Thread-Safe	Not Inherently Thread-Safe	Modern Threading Model

(b) What is the level of hierarchy of java AWT 2

Solve:
The hierarchy of Java AWT is based on the following levels:
1. Component: The root class for all AWT components.
2. Container: Extends Component and serves as a base class for components that can contain other components.
3. Window: Extends Container and represents a top-level window.
4. Frame: Extends Window and represents a decorated window with title and border.
5 .Dialog: Extends Window and represents a top-level window with a title and a blocking behavior.
6. Button, TextField, Label, etc.: Various components that can be added to containers.

This hierarchy allows the creation of graphical user interfaces in Java AWT

(c) Write a program to create three buttons with caption OK, SUBMIT, CANCEL 4



Solve:

```
import java.awt.Button;
import java.awt.Frame;
import java.awt.FlowLayout;

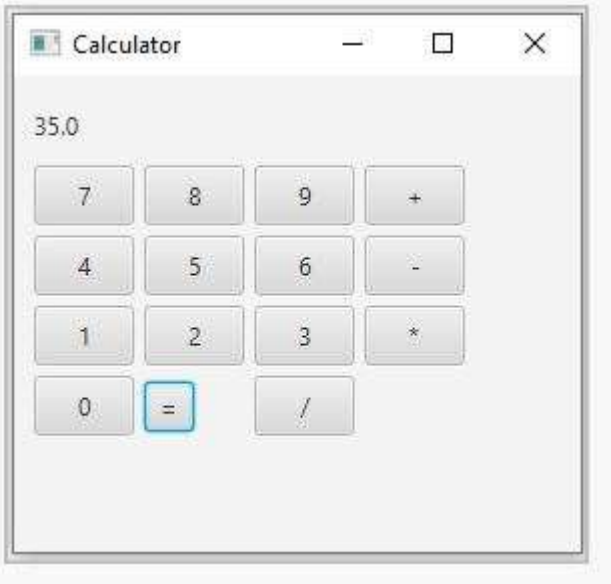
public class ButtonExample {
    public static void main(String[] args) {
        Frame frame = new Frame("Button Example");
        Button okButton = new Button("OK");
        Button submitButton = new Button("SUBMIT");
        Button cancelButton = new Button("CANCEL");

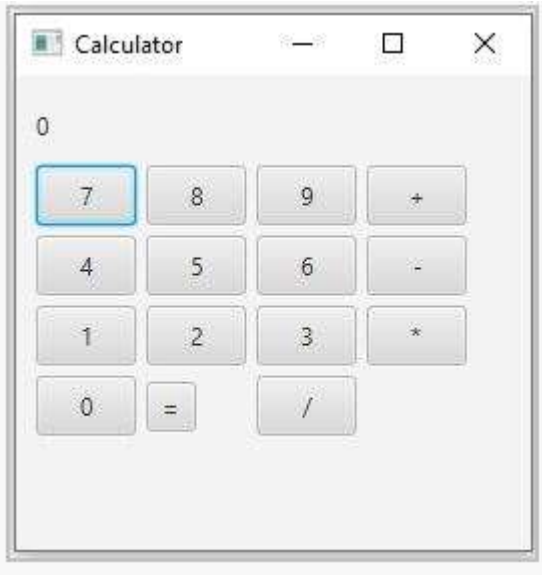
        frame.setLayout(new FlowLayout());
        frame.add(okButton);
        frame.add(submitButton);
        frame.add(cancelButton);

        frame.setSize(300, 100);
        frame.setVisible(true);
    }
}
```

(8)

(a) Write a JavaFX program to build a simple calculator with digit buttons (0-9) and basic arithmetic operator buttons (+, -, *, /). Implement action listeners for each button (digits and operators) to perform calculations. 10





Solve: import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.GridPane;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class Calculator extends Application {
    private Text display = new Text("");
    private StringBuilder input = new StringBuilder();

    public static void main(String[] args) {
        launch(args);
    }

    @Override
    public void start(Stage stage) {
        GridPane gridPane = new GridPane();
        gridPane.setHgap(5);
        gridPane.setVgap(5);

        for (int i = 0; i < 10; i++) {
            Button digitButton = createDigitButton(Integer.toString(i));
            gridPane.add(digitButton, i % 3, i / 3);
        }

        String[] operators = {"+", "-", "*", "/"};
        for (int i = 0; i < operators.length; i++) {
            Button operatorButton = createOperatorButton(operators[i]);
            gridPane.add(operatorButton, 3, i);
        }

        Button equalsButton = createEqualsButton();
        gridPane.add(equalsButton, 2, 4);
        Button clearButton = createClearButton();
        gridPane.add(clearButton, 3, 4);

        gridPane.add(display, 0, 0, 3, 1);

        Scene scene = new Scene(gridPane, 200, 250);
        stage.setTitle("Simple Calculator");
        stage.setScene(scene);
        stage.show();
    }

```
private Button createDigitButton(String digit) {
    Button button = new Button(digit);
    button.setOnAction(e -> {
        input.append(digit);
        display.setText(input.toString());
    });
    return button;
}

private Button createOperatorButton(String operator) {
    Button button = new Button(operator);
    button.setOnAction(e -> {
        input.append(" ").append(operator).append(" ");
        display.setText(input.toString());
    });
    return button;
}

private Button createEqualsButton() {
    Button button = new Button("=");
    button.setOnAction(e -> {
        try {
            String result = evaluateExpression();
            display.setText(result);
            input.setLength(0);
            input.append(result);
        } catch (Exception ex) {
            display.setText("Error");
            input.setLength(0);
        }
    });
    return button;
}

private Button createClearButton() {
    Button button = new Button("C");
    button.setOnAction(e -> {
        input.setLength(0);
        display.setText("");
    });
    return button;
}
```

```
    private String evaluateExpression() {
        return String.valueOf(eval(input.toString()));
    }

    private double eval(String expression) {
        return new Object() {
            int pos = -1, ch;

            void nextChar() {
                ch = (++pos < expression.length()) ? expression.charAt(pos) : -1;
            }

            boolean isDigit(char c) {
                return Character.isDigit(c);
            }

            double parse() {
                nextChar();
                double x = parseExpression();
                if (pos < expression.length()) throw new RuntimeException("Unexpected: " + (char)
ch);
                return x;
            }

            double parseExpression() {
                double x = parseTerm();
                while (true) {
                    if (eat('+')) x += parseTerm();
                    else if (eat('-')) x -= parseTerm();
                    else return x;
                }
            }

            double parseTerm() {
                double x = parseFactor();
                while (true) {
                    if (eat('*')) x *= parseFactor();
                    else if (eat('/')) x /= parseFactor();
                    else return x;
                }
            }

            double parseFactor() {
                if (eat('+')) return parseFactor();
```

```
            if (eat('-')) return -parseFactor();

            double x;
            int startPos = this.pos;
            if ((ch >= '0' && ch <= '9') || ch == '.') {
               while ((ch >= '0' && ch <= '9') || ch == '.') nextChar();
               x = Double.parseDouble(expression.substring(startPos, this.pos));
            } else if (ch >= 'a' && ch <= 'z') {
               while (ch >= 'a' && ch <= 'z') nextChar();
               String func = expression.substring(startPos, this.pos);
               x = parseFactor();
               if (func.equals("sqrt")) x = Math.sqrt(x);
               else throw new RuntimeException("Unknown function: " + func);
            } else {
               throw new RuntimeException("Unexpected: " + (char) ch);
            }

            return x;
         }

         boolean eat(char charToEat) {
            while (ch == ' ') nextChar();
            if (ch == charToEat) {
               nextChar();
               return true;
            }
            return false;
         }
      }.parse();
   }
}
```